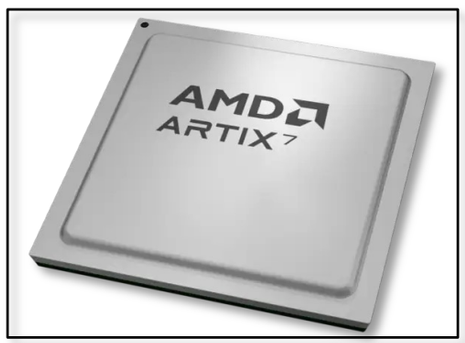
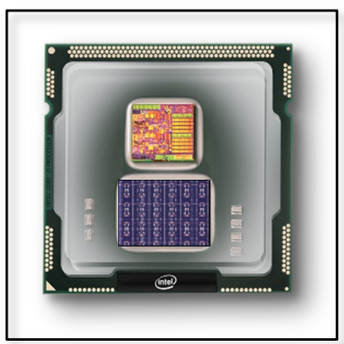
Christof Teuscher

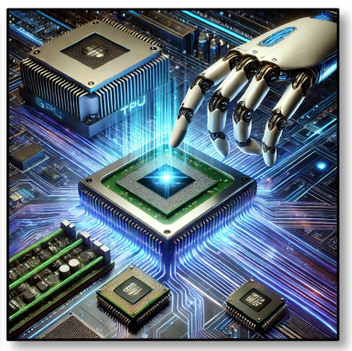
ECE 410/510: Hardware for AI and ML, Spring 2026

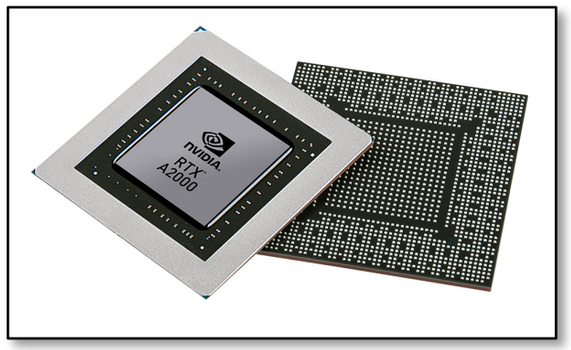
VLSI Design + EDA Tools

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu









teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

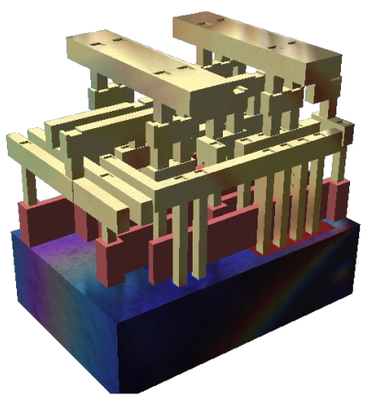
VLSI Design Overview

- **VLSI** (Very Large Scale Integration) places billions of transistors on a chip.
- **ASIC** (Application-Specific Integrated Circuit) is a specific type of IC designed for a custom, dedicated function. **VLSI is the field, while ASICs are the end product.**
- Design is hierarchical: high-level behavior → gates → transistors → layout
- Each abstraction level is captured and verified before moving down.
- **Electronic Design Automation** (EDA) tools automate synthesis, place-and-route, and verification.
- Modern AI/ML chips follow this same flow. You are designing an ASIC by using VLSI + EDA tools.

Design Abstraction Levels

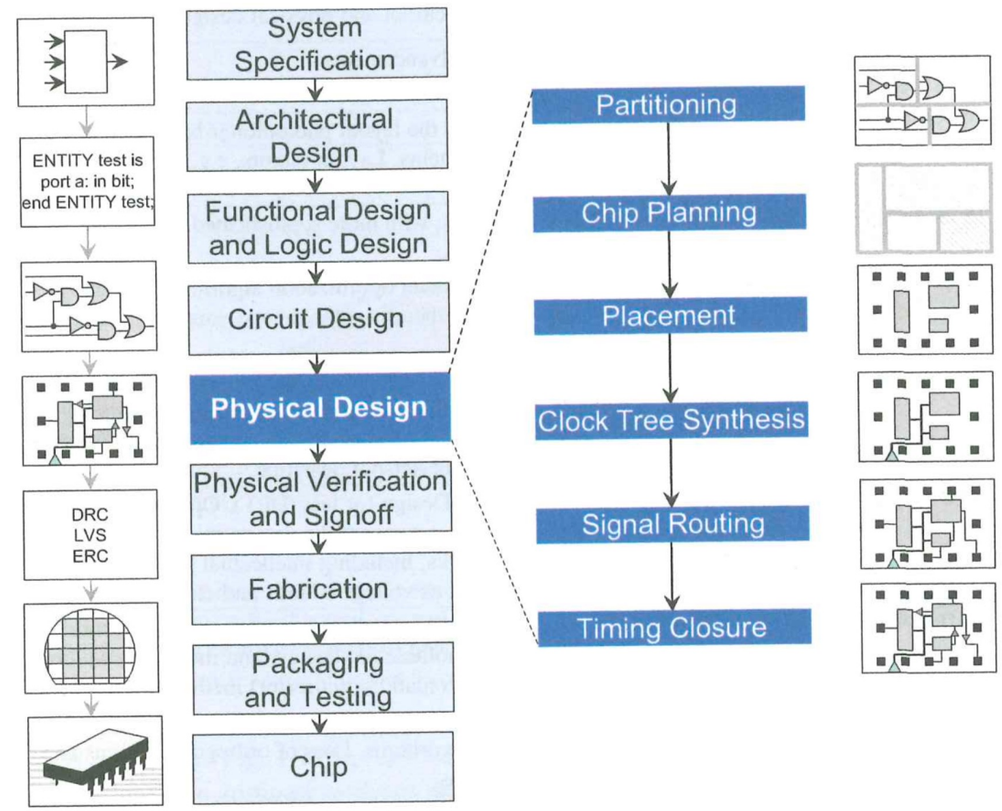
Abstraction levels (top to bottom)

- Algorithmic / Python
- RTL / Verilog (SystemVerilog)
- Gate Level (Netlist)
- Physical Layout (GDSII)
- Transistor Level (Silicon)



A rendering of a small GDSII standard cell with three metal layers. The sand-colored structures are metal interconnect, with the vertical pillars being contacts, typically plugs of tungsten. The reddish structures are polysilicon gates, and the solid at the bottom is the crystalline silicon bulk.

https://en.wikipedia.org/wiki/GDSII



Kahng, Andrew B., Jens Lienig, Igor L. Markov, and Jin Hu. **VLSI physical design: from graph partitioning to timing closure**. Vol. 312. Netherlands: Springer, 2011.

Key EDA Tools

Simulation & Verification

- Icarus Verilog / Verilator — open-source RTL simulators
- cocotb — Python-based testbench framework (used in this course)
- ModelSim / QuestaSim — industry-standard simulation

Synthesis

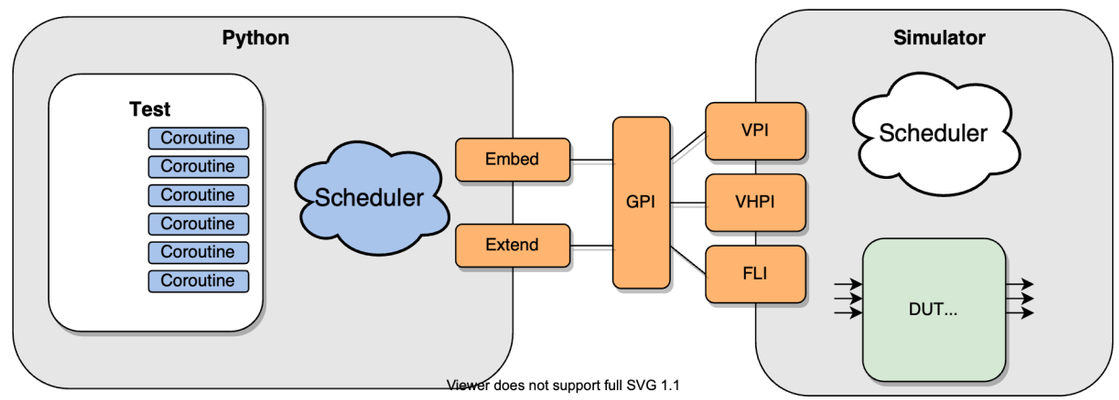
- Yosys — open-source logic synthesis
- Synopsys Design Compiler / Cadence Genus — industry tools

Place, Route & Physical Verification

- OpenROAD — open-source place-and-route (used in OpenLane flow)
- Cadence Innovus / Synopsys IC Compiler — industry P&R
- Magic / KLayout — layout viewing and Design Rule Check (DRC)

Introduction to cocotb

- **cocotb = Coroutine-based Co-simulation Test Bench**
- Allows writing hardware testbenches in Python, not Verilog.
- Interfaces with HDL simulators (Icarus, Verilator, ModelSim) via VPI/VHPI
- Uses Python async/await: tests run as coroutines in simulation time
- Leverage the full Python ecosystem: NumPy, PyTorch, pytest, logging
- Open source (Apache 2.0), actively maintained, widely used in industry



cocotb Design Flow

Typical cocotb workflow

1. Write Design under Test (DUT) in Verilog (or VHDL)
2. Write Python testbench using cocotb decorators. Ask your LLM to write testbenches!
3. Configure Makefile: select simulator, top module.
4. Run: make → simulator launches → cocotb loads tests.
5. Python drives signals, reads outputs, asserts correctness.
6. View waveforms (GTKWave) and fix RTL bugs.

Key concepts

- @cocotb.test(): marks an async test function
- dut.signal.value : read/write DUT ports
- await RisingEdge(dut.clk): synchronize to clock
- Clock() helper: auto-generates clock signal
- pytest integration for structured test reporting
- VCD/FST waveform dumps for GTKWave debugging

Your AI Co-Processor Design Flow

Phase 1: Algorithm → Python Model

- Implement AI operator in Python (e.g., matrix multiply, convolution)
- Python model serves as golden reference for verification

Phase 2: RTL Design in Verilog

- Design synthesizable Verilog modules (MAC units, buffers, controllers)
- Follow microarchitecture spec provided in assignments

Phase 3: Verify with cocotb

- Write Python tests comparing RTL output to Python golden model
- Test with random inputs, edge cases, and full inference runs

Phase 4: Synthesis & Reporting

- Run Yosys or OpenLane for area/timing estimates
- Report throughput, latency, area, and power trade-offs

Evaluating Your AI Accelerator (Simulation-Only)

Step 1: Get Max Clock Frequency

- OpenLane timing report (OpenSTA): read Worst Negative Slack (WNS)
- **Fmax = 1 / (T_clk − WNS)**
- Report Fmax in MHz; compare to TT board clock (≈50 MHz default)

Step 2: Measure Latency in Simulation

- In cocotb testbench: record start/end cycle for one inference (e.g., MAC array, ReLU pass)
- **Latency = N_cycles / Fmax**
- Report pipelined throughput.

Step 3: Area & Power from OpenLane

- Area: total cell area (μm²) and utilization % from metrics.csv or summary report
- Power: static + dynamic estimates from OpenSTA power analysis (mW @ Fmax)
- VCD-based switching activity from cocotb sim gives more accurate dynamic power

Step 4: Efficiency & Accuracy Metrics

- **Throughput (GOPs) = (ops/inference × Fmax) / 10^9**
- **Efficiency = GOPs / W and GOPs / mm²**
- Accuracy: compare fixed-point HW output vs. Python float reference in cocotb (bit-exact or MSE)

Performance Reporting Scorecard

Core Performance Metrics

- **Fmax (MHz)** → OpenSTA WNS report
- **Latency (cycles / ns)** → cocotb cycle counter
- **Throughput (GOPs)** → ops/infer × Fmax
- **Area / Power / Efficiency** → OpenLane metrics.csv + OpenSTA
- **Accuracy (MSE / bit-exact %)** → cocotb vs. Python float ref

Interface Metrics (M1–M4 Required)

- **Interface type + peak BW** → spec (SPI/I²C/AXI/PCIe/UCIe)
- **Effective BW vs. kernel demand** → roofline: interface-bound?
- **Transaction overhead (cycles)** → cocotb protocol sim
- **End-to-end latency (host → result)** → full co-sim pipeline

What to Include in Your Report

✓ OpenSTA timing summary screenshot (WNS → Fmax)
✓ cocotb sim log: cycle count + waveform (VCD/GTKWave)
✓ Completed metrics table (all 7 rows filled)
✓ Fixed-point HW vs. Python float comparison (plot or table)
✓ Bottleneck discussion: timing-limited / area-limited / I/O-limited?

Example (8-bit 4×4 MAC array, 1 tile):

→ Fmax = 80 MHz, Latency = 12 cyc (150 ns), 16 MACs/cycle → 2.56 GOPs
→ Area = 9,800 μm², Power ≈ 3.2 mW → 800 MOPs/W, MSE < 1 LSB

Python → Accelerator: Interface Hooks

The Host Driver Pattern

- **Write a Python driver class with methods matching your interface**: write_reg(), send_tensor(), recv_result()
- **In simulation**: methods call a **Bus Functional Model** (BFM) coroutine (cocotbext-axi / cocotbext-spi) that drives pins cycle-accurately alongside your DUT

BFM Examples (cocotbext library)

- **SPI:** await spi.write(bytes(weights)); r = await spi.read(4)
- **AXI4-Stream:** await src.send(AxiStreamFrame(data)); f = await sink.recv()
- **AXI4-Lite:** await axil.write(addr, data); val = await axil.read(addr)

Replacing the SW Kernel with HW

1. Profile Python; wrap dominant kernel: hw_matmul(A,B) calls driver.send_tensor()
2. Quantize inputs to HW precision (INT8/INT16); dequantize outputs on return
3. In co-sim: record cocotb.utils.get_sim_time() at send and result-valid; compute simulated T_total
4. Compare hw_matmul() output vs. np.matmul() reference — check both correctness and latency

Interface Metrics to Report

- **T_transfer** = bytes_sent / effective_BW — compare to T_compute to find bottleneck
- **Transaction overhead**: idle cycles between writes, visible in GTKWave waveform

Christof Teuscher • teuscher@pdx.edu

Full-System Benchmarking in Co-Simulation

The Full Co-Simulation Pipeline

1. **Python host** prepares input tensor, records t_start (wall clock)
2. **Interface BFM** drives SPI/AXI transactions; DUT clock cycles tick in Verilator/Icarus
3. **Accelerator core** computes; BFM reads result back; Python records sim time (get_sim_time)
4. **Simulated time** (ns) × Fmax → estimated real-silicon latency and throughput

Time Breakdown to Report

- **T_transfer** = cycles from first interface txn to last byte received by DUT
- **T_compute** = cycles from data-ready to result-valid signal asserted
- **T_total** = T_transfer + T_compute + T_readback — vs. Python SW baseline time

teuscher Lab
teuscher-lab.com

SW Baseline vs. HW Accelerated (M4)

- SW baseline: time Python kernel (time.perf_counter) on your laptop CPU — record from M1
- HW estimate: T_total (sim ns) × Fmax_ratio → projected latency at silicon speed
- Speedup = T_SW / T_HW — if <1×, find out whether transfer or compute dominates

Roofline Update for M4

- Re-plot kernel on roofline with actual Fmax, measured BW (interface), and achieved GOPs
- Show two ceilings: peak compute (GOPs @ Fmax) and interface BW ceiling (GB/s × arith. intensity)
- Explain any gap between achieved and theoretical peak — the bottleneck analysis is graded

Portland State UNIVERSITY

Example comparison (SPI, 8-bit MAC):

Christof Teuscher • teuscher@pdx.edu

Python to Verilog Translation Guide

Python (Behavioral)

```python
import numpy as np

def mac(a, b, acc):
    # Multiply-Accumulate
    return acc + a * b

def relu(x):
    return max(0, x)

# 8-bit fixed point
a, b = np.int8(3), np.int8(5)
result = mac(a, b, 0)
```

Synthesize →

SystemVerilog (RTL)

```verilog
module mac #(parameter W=8) (
    input  logic signed [W-1:0] a, b,
    input  logic signed [15:0]  acc,
    output logic signed [15:0]  result
);
    // Multiply-Accumulate
    assign result = acc + (a * b);
endmodule

// ReLU: check sign bit
assign relu_out = x[W-1] ? 0 : x;
```

Python Concept → Verilog Equivalent

- def func(a, b) → module + port list
- int / np.int8 → logic [7:0]
- if/else → always_comb if / ternary assign
- for loop → generate or always_ff
- return val → assign output port

Synthesis Rules — Avoid Pitfalls

- Declare all bitwidths explicitly — no dynamic sizing
- Use always_ff for registers; always_comb for logic
- Avoid *, / and % — use bit shifts or pre-computed LUTs
- Parameterize bitwidths — no magic numbers
- Run Verilator lint before submitting to synthesizer

linting = statically analyzing your design for issues

Christof Teuscher • teuscher@pdx.edu

Synthesizing Python: The Tool Chain

You cannot synthesize plain Python directly. You must write in a Python HDL Domain-Specific Language (DSL) that emits synthesizable RTL.

Python HDL DSL	→	Auto-Conversion	→	RTL Simulator / Linter	→	Synthesizer
MyHDL / PyMTL3 / Amaranth		toVerilog() or build system		Verilator / Iverilog lint		Yosys or Synopsys DC

MyHDL

`pip install myhdl`

- Oldest Python HDL (2003)
- toVerilog() / toVHDL() built-in
- @always_comb, @always_seq decorators
- intbv for fixed-width integers
- Simulates and converts in one step

PyMTL3

`pip install pymtl3`

- Cornell / MIT lineage
- Class-based modules (Component)
- update() / update_ff() blocks
- Strong multi-level modeling support
- Emits clean SystemVerilog

Amaranth HDL

`pip install amaranth`

- Modern Python 3 syntax (nMigen successor)
- m.d.comb += / m.d.sync += style
- Built-in Yosys synthesis backend
- Supports FPGA place-and-route
- Active community, Tiny Tapeout-ready

Tiny Tapeout: Chips for Everyone

What is Tiny Tapeout?

- Shared-silicon ASIC shuttle makes real chip fabrication affordable for students.
- Many designs on one chip; each gets a tile (~160×100 µm, ~1000 gates).
- SkyWater 130nm open PDK + OpenLane for fully open-source ASIC flow.
- 24 I/O pins per design: 8 in, 8 out, 8 bidirectional; ≥50 MHz clock.
- Timeline: submission to chip delivery ≈ 6–12 months.

Where It Fits in the VLSI Flow

- Your Verilog RTL is the entry point, same code tested with cocotb
- GitHub Actions CI runs OpenLane automatically: synthesis → P&R → GDSII
- No manual Yosys/OpenROAD setup — the TT template handles it all.
- Closes the loop: Python → Verilog → cocotb → Tiny Tapeout → real silicon.
- Buy extra tiles if your AI co-processor exceeds single-tile gate count

Great resources, docs, workshops, etc: https://tinytapeout.com

Tiny Tapeout Calculator

Shuttle: SKY130 (CHIPFOUNDRY) | IHP | GLOBALFOUNDRIES

This shuttle is run by wafer.space.

Who are you? ACADEMIC/INDUSTRY | INDIVIDUAL

Number of tiles

1

Number of PCBs

1

Pricing

Item	Qty	Total
Tile Cost	1	70 €
DevKits (PCBs)	1	~~300 €~~ 100 €
Worldwide Economy Shipping	-	15 €
Total to pay		~~385 €~~ **185 €**

Christof Teuscher • teuscher@pdx.edu

Tiny Tapeout: Student Submission Steps

Design & Verify

1. Fork the TT GitHub template repository
2. Write your AI co-processor in Verilog (src/ folder); map I/O to TT pinout (ui_in, uo_out, uio)
3. Write cocotb testbench; run make SIM=icarus to verify correctness
4. Fill in info.yaml: title, description, how_it_works, how_to_test, I/O table

Automated Build

5. Push to GitHub → Actions run OpenLane automatically → GDS and Docs checks must pass
6. Review layout preview and area/timing reports in the Actions artifacts

Submit & Receive Silicon

7. Log in to app.tinytapeout.com → create project → paste GitHub repo URL
8. Pay for a tile slot (or use class-provided prepaid credits); shuttle deadline applies
9. Wait 6–12 months → receive demo PCB with your chip and all others on the shuttle

Key Constraints to Plan For

- Gate budget: ~1000 gates/tile, ops small or buy extra tiles
- I/O pins are limited, use serial interfaces or on-chip memory for wide data paths
- Cocotb testbench testing = required; Tiny Tapeout template includes cocotb by default